

Programming 3A

Student Numbers:	ST10440685
Programme Code:	BCAD0701
Module Lecturer:	Adam Mortda
Module Code:	PROG7311
Date of Submission:	27-03-2026

I hereby declare that I did not plagiarise the content of this assignment and that this is my own work.

Assignment submitted via SafeAssign: ✓ (Tick the Box)

Contents

Introduction	2
Enterprise Architecture Framework	2
System Architecture Design	3
TOGAF Diagram	4
Design Patterns	4
UML Diagram	5
Non Functional Requirements	6
Availability	6
Interoperability	6
Scalability	7
Reference:	8

INTRODUCTION

The Global Logistics Management System purpose is to modernize TechMove Logistics' outdated operational systems. The organization currently relies on disjointed spreadsheets, emails, and manual communication. This results in the organization experiencing inefficiencies, data inconsistencies, and compliance risks.

The objective of this project is to deliver an enterprise grade, scalable, and highly available web based system. This system will be capable of managing contracts, service requests, and financial integrations. This proposed solution will implement architectural principles, which includes service oriented architecture and containerization. This ensures flexibility, performance, and future growth.

ENTERPRISE ARCHITECTURE FRAMEWORK

The GLMS architecture will be structured using The Open Group Architecture Framework (TOGAF). This provides the project with a systematic approach for designing, planning, and implementing enterprise information architecture. TOGAF provides a blueprint for aligning business strategy with technology (CIO, 2022). This ensures that the organization's IT infrastructure supports and meets the goals and objectives. Establishing a clear framework allows Enterprise Architecture facilitates better decision making and enhanced agility in responding to the changes in a business environment (CIO, 2022).

The Architecture Development Method (ADM) will guide the system development by:

- Phase A (Architecture Vision) – The overall vision of GLMS is defined, mainly focusing on efficiency, automation, and integration improvement (Visual Paradigm, 2024).
- Phase B (Business Architecture) – The key business processes such as contract management, and financial operations are identified (Visual Paradigm, 2024)
- Phase C (Information Systems Architecture) – The data and application architecture is created, this includes databases, and services (Visual Paradigm, 2024).

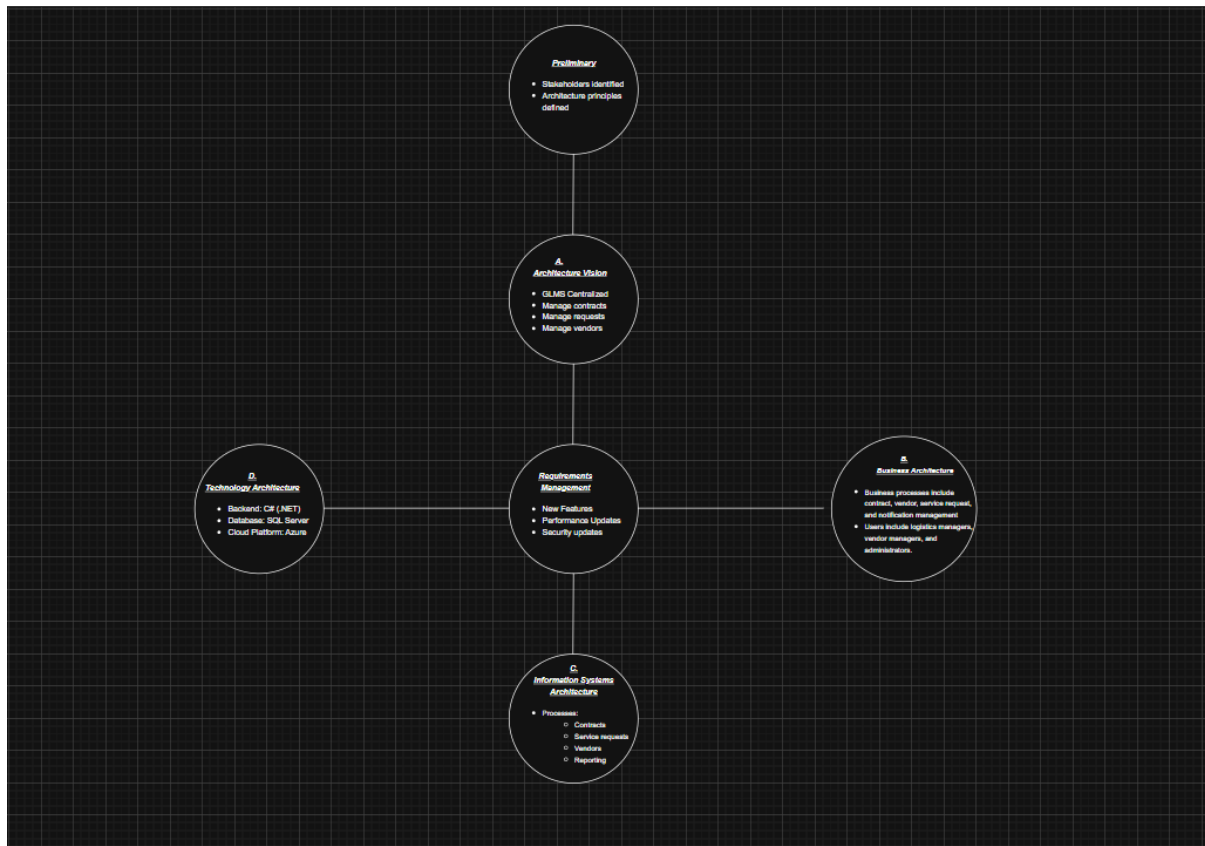
- Phase D (Technology Architecture) – The technology infrastructure required to support the architecture is defined. This includes cloud infrastructure and micro services (Visual Paradigm, 2024).
- Phase E & F (Opportunities and Migration Planning) - The transition from the current system to the new system is planned (Visual Paradigm, 2024).
- Phase G (Implementation Governance) – Proper development and deployment practices must be ensured (Visual Paradigm, 2024).
- Phase H (Architecture Change Management) – Changes will need to be managed over time. This supports continuous updates and scalability (Visual Paradigm, 2024).

SYSTEM ARCHITECTURE DESIGN

The GLMS will implement service orientated architecture (SOA) with containerized micro services. The system will consist of new components such as:

- Frontend: Logistics managers and admins will have a web based user interface (IIE, 2026).
- API Gateway: This will handle all the incoming requests and redirect them to the correct services (IIE, 2026).
- Micro services – Contract service, Service Request service, and Financial service (IIE, 2026).
- Database layer: Storing contracts, users, and transactions will be stored in centralized and distributed databases (IIE, 2026).
- External APIs: Currency conversions are required for the financial integration (IIE, 2026).
- Docker Containers: Each service runs independently to prove the scalability and deployment (IIE, 2026).

TOGAF Diagram



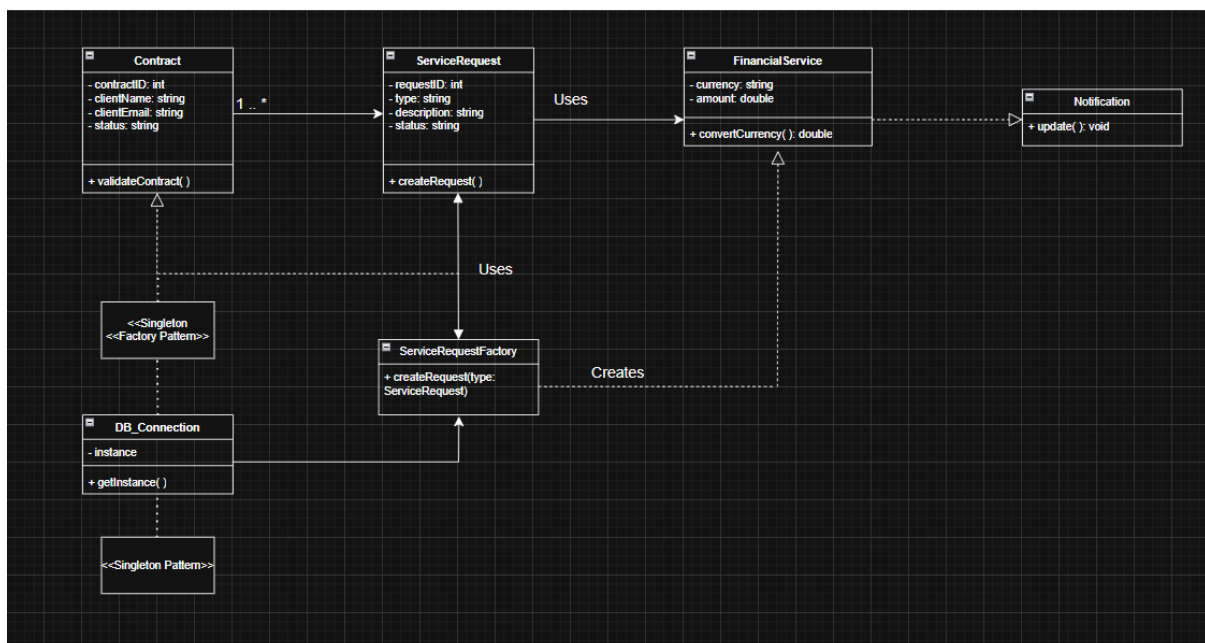
DESIGN PATTERNS

The three design patterns that will be implemented are:

- **Singleton Pattern** – This will ensure that the system will have a single instance of database connection and it provides a global point of access to it. This improves the overall efficiency when managing shared resources like the database connections (SourceMaking, 2018). Multiple services will require access to the database and creating multiple connections could cause performance issues and resource exhaustion. Implementing Singleton design pattern will be used for the Database Connection class. This ensures only one database connection is created and all the services share the same class. This decreases memory usage and the system's performance improves (SourceMaking, 2018).

- Abstract Factory Pattern – This design pattern is used to create service request objects dynamically without the need to specify the exact classes. This assists with flexibility and reduces tight coupling between classes. Instead of creating these objects manually by using “new”, the system will use a ServiceRequestFactory class. This class will generate the appropriate request type dynamically (SourceMaking, 2018).
- Observer Pattern – This design pattern allows objects to notify other objects automatically when there are changes. This is a vital tool for implementing real time updates and event driven systems (SourceMaking, 2018). This design pattern will be used for the notifications and alerts like a contract expiring warning in the GLMS. Implementing this design pattern in the GLMS will improve the real time communication and the overall system responsiveness. This improves the user satisfaction of using the system (SourceMaking, 2018).

UML Diagram



NON FUNCTIONAL REQUIREMENTS

Availability

Availability refers to the ability of the system to remain operational and accessible when required. The GLMS must be able to ensure 24/7 availability. This ensures that the logistics managers, clients, and drivers will have access to the system at any time (GeeksforGeeks, 2024).

The system will need to implement cloud based hosting, redundant servers so that downtime can be prevented, and load balancing to prevent the server experiencing an overload. If one server hosting the Contact service fails, another instance will take over automatically. This ensures that the organization experiences minimal downtime and business continuity is maintained. This also improves the user's trust in the system (IIE, 2026).

Interoperability

Interoperability refers to the ability of the system to integrate with other systems and services in a coordinated manner without effort from the end user. Interoperability assists organizations to achieve higher efficiency and a more holistic view of information (Lewis, 2023).

The GLMS has to be able to integrate with APIs for currency conversions, financial systems, legacy systems, and third party logistics. Generating invoices, the system will have to retrieve currency conversion rates from external APIs at real time. Implementing interoperability into the system will ensure that integration is seamless and manual work is reduced (Lewis, 2023).

SCALABILITY

Horizontal scaling refers to the process of increasing the amount and performance of a system by adding more servers or machines to distribute the workload across the number of individual units. Instead of relying on a single server to process the new contracts, multiple servers will need to be deployed. These servers work together and divide the workload. This ensures no server experiences overload. This improves the systems reliability and performance (GeeksforGeeks, 2026).

Load Balancing refers to distributing incoming traffic across multiple servers. This helps to prevent overload and prevent system bottlenecks. Load balancing distributes the incoming requests evenly, ensuring that response times are improved (Yasar, 2023). If many users are accessing the contracts or creating service requests at the same time, the load balancer ensures that each server handles its share of requests. This ensures that the system runs smoothly during peak usage (Yasar, 2023).

Caching refers to storing frequently accessed data in temporary memory. This helps to reduce database load. The GLMS can cache the contract data, user information, and reports that are accessed frequently. Frequently accessed contracts are stored in cache for faster access. When another user requests the same data, the system can retrieve the data from the cache instead of querying the database. This reduces the database load and improves the systems performance (Singh, 2025).

REFERENCE:

1. IIE, 2026. Programming 3A PROG7311 Enterprise Application Development EAPD7111 Module Manual 2026 (First Edition: 2016), 2026. [Online]. Available at: https://advtechonline.sharepoint.com/:w:/r/sites/TertiaryStudents/_layouts/15/Doc.aspx?sourcedoc=%7B3870E594-2BBD-47BB-8E7E-FDDC61AFEACD%7D&file=PROG7311MM.docx&action=default&mobileredirect=true. [Accessed 24 March 2026].
2. Visual Paradigm, 2024. Understanding Enterprise Architecture (EA) and TOGAF, 2024. [Online]. Available at: [Understanding Enterprise Architecture \(EA\) and TOGAF - Visual Paradigm Guides](#). [Accessed 24 March 2026].
3. White S.K., 2022. What is TOGAF? What the enterprise architecture methodology means for your business, 2022. [Online]. Available at: [What is TOGAF? What the enterprise architecture methodology means for your business | CIO](#). [Accessed 24 March 2026].
4. GeeksforGeeks, 2026. Gang of Four (GOF) Design Patterns, 2026. [Online]. Available at: [Gang of Four \(GOF\) Design Patterns - GeeksforGeeks](#). [Accessed 24 March 2026].
5. SourceMaking, 2018. Singleton Design Pattern, 2018. [Online]. Available at: [Singleton Design Pattern](#). [Accessed 25 March 2026].
6. SourceMaking, 2018. Observer Design Pattern, 2018. [Online]. Available at: [Observer Design Pattern](#). [Accessed 25 March 2026].
7. SourceMaking, 2018. Abstract Factory Design Pattern, 2018. [Online]. Available at: [Abstract Factory Design Pattern](#). [Accessed 25 March 2026].
8. GeeksforGeeks, 2026. Unified Modeling Language (UML) Diagrams, 2026. [Online]. Available at: [Unified Modeling Language \(UML\) Diagrams - GeeksforGeeks](#). [Accessed 25 March 2026].
9. GeeksforGeeks, 2024. What are Non-Functional Requirements in System Design? | Examples, Definition, 2024. [Online]. Available at: [What are Non-Functional Requirements in System Design? | Examples, Definition - GeeksforGeeks](#). [Accessed 25 March 2026].
10. Lewis S., 2023. Interoperability, 2023. [Online]. Available at: [What is interoperability? | Definition from TechTarget](#). [Accessed 25 March 2026].

11. GeeksforGeeks, 2026. Horizontal and Vertical Scaling | System Design, 2026. [Online]. Available at: [Horizontal and Vertical Scaling | System Design - GeeksforGeeks](#). [Accessed 25 March 2026].
12. Singh M., 2025. What Is Caching?, 2025. [Online]. Available at: [What Is Caching? \(Definition, How It Works, Types\) | Built In](#). [Accessed 25 March 2026].
13. Yasar K., 2023. Load Balancing, 2023. [Online]. Available at: [What is Load Balancing? How Load Balancing Works | TechTarget](#). [Accessed 25 March 2026].